SmartCamra Project

From Smart Camera Module to Self-Navigation RC-Car

SangWoo Park, Hantao Fu, Mehul Goel
Electrical & Computer Engineering, Rice University, Houston, TX 77005 USA

***Abstract*–Autonomous driving has emerged as one of the most transformative technologies in recent years. However, most academic and open-source implementations remain constrained to controlled indoor environments or predefined track settings. This project addresses the critical challenge of transitioning autonomy from controlled environments to real-world, dynamic outdoor spaces. We introduce *SmartCam*, an autonomous RC car capable of robust navigation across the sidewalks and pathways of Rice University. Building upon previous RC-car autonomy frameworks [1], [2], our system integrates real-time localization [3], visual perception for obstacle avoidance, and robust motion planning to achieve reliable operation in dynamic outdoor conditions on campus pathways [4]. The system was successfully tested in a complex takeout delivery scenario, demonstrating reliable path execution and adaptation to environmental variations such as lighting and surface irregularities. The framework provides a functional, modular platform for developing self-driving capabilities in challenging outdoor settings.**

***Index Terms*—Autonomous driving, Computer vision, Path planning, Embedded AI, Smart mobility, Edge computing.**

I. Introduction

A growing number of open-source and academic initiatives have explored autonomous miniature vehicles [1], [2], [5], [6]. Despite their contributions to perception and control research, most implementations remain confined to indoor test tracks or controlled lighting environments. Consequently, these systems do not fully address the complexities of outdoor navigation, such as variable illumination, dynamic obstacles, and the need for reliable localization under GPS-denied or noisy conditions. To overcome these limitations, this project aims to extend the autonomous RC car framework toward robust outdoor navigation on the Rice University campus. The proposed *SmartCam* platform initially sought to integrate object detection and Simultaneous Localization and Mapping (SLAM) within a unified software stack. However, due to significant sensor instability and error accumulation inherent to the single RC-car chassis when operating outdoors, the system required an innovative fundamental shift. The final system was successfully reframed into a Leader-Follower architecture, separating the demanding localization task (performed by a dedicated Leader car using Lidar SLAM) from the heavy computational and payload task (performed by the main Follower RC car using computer vision-based ArUco code tracking). This dual-vehicle design achieved stable navigation and culminated in the successful demonstration of an End-to-End Takeout Delivery Task in a real-world outdoor campus environment. The project thus provides a robust and scalable solution for deploying autonomous platforms in challenging dynamic environments.

II. Methodology

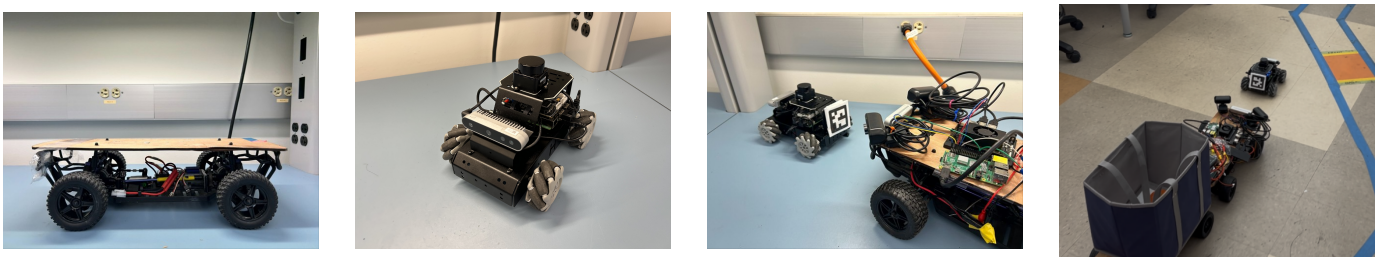
A. General Structure

i. Setbacks of the Initial Single-Car Platform: During the early stage of the project, we devised a solution utilizing a simple RC-Car (As shown in Fig. 1a) for navigation and carrying goods. Unfortunately, this platform's structure impeded effective mapping. A primary reason for this difficulty was the lack of odometry sensors, which are the most critical component for Lidar-based Simultaneous Localization and Mapping (SLAM). Crucially, the simple RC-Car used in this project was not equipped with such sensors.

The odometry deficiency presents two potential solutions. The first is utilizing an Inertial Measurement Unit (IMU), such as the MPU6050, which estimates distance through the double-integration of acceleration data. However, the resulting accumulating error often makes the mapping process unstable and inaccurate. The second option is vision-based SLAM, where the RC-car determines its position by sensing feature points from a camera's video stream. Nevertheless, such algorithms typically involve higher computational cost, thereby weakening the system's real-time processing capability. Concurrently, feature-point-based odometry often necessitates a large dataset for accurate recognition, requiring significantly more storage space than Lidar-based SLAM.

ii. Leader-Follower Plan: Considering the constraints of time and resources, we adopted the Hiwonder robot car (As shown in Fig. 1b) for mapping and navigation. The Hiwonder robot car is highly integrated with multiple sensors, including an STL-19P TOF Lidar for mapping, a hall encoder geared motor for sensing position changes (odometry), a 3D depth camera for computer vision, and an RRC Lite Controller for real-time processing. This integration, along with detailed tutorial materials and extensive example source code, made mapping and navigation significantly easier compared to a DIY or self-assembled solution.

However, the single Hiwonder robot car also faces setbacks. A major drawback is its limited battery capacity, which hinders its ability to carry or tow heavy loads. Furthermore, its limited expandability prevents the straightforward addition of transportation accessories, as the car is densely packed with sensors on its mechanical structure. This highly integrated property



(a) RC-Car:Follower (b) Hiwonder:Leader (c) Follwer-Leader. (d) Final Demo:Delivery.

Fig. 1: Overview reflecting the development stages: (a) used for basic component tests and serving as the Follower vehicle. (b) selected as the Leader platform due to its integrated SLAM and odometry capabilities. (c) showing the ArUco code on the leader being tracked by the follower. (d) is verifying the system's operational reliability.

contrasts with the simple RC-Car used in the previous single-car plan, where a wood board above the platform ensured better expandability.

To address the disadvantages of both the simple RC-Car and the Hiwonder robot car, we implemented a Leader-Follower structure (As shown in Fig.1c). In this architecture: The Hiwonder robot car performs the navigation task, serving as the Leader to determine the route. The Simple RC-Car carries heavy goods, functioning as the Follower by chasing the Leader.

The overall system thus successfully integrates both accurate navigation and heavy-load capabilities. Additionally, since the number of follower RC-cars can be scaled up, this structure provides significantly greater expandability than the single-car plan.

B. Follower RC-Car Control System

Table III(Appendix) and Fig. 2 illustrate the main components and control logic of the Follower RC-car.

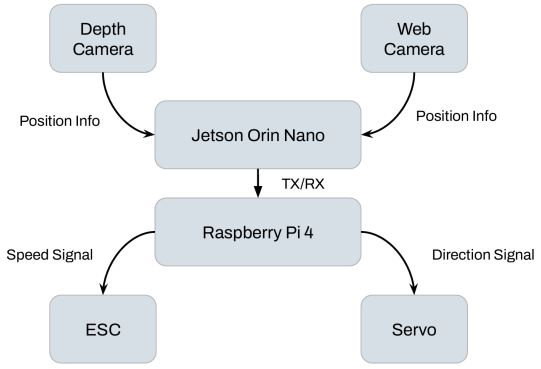
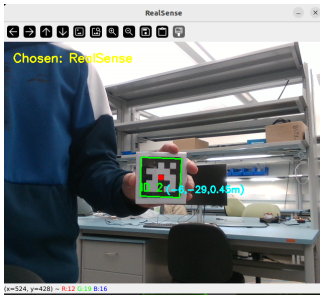


Fig. 2: The Controlling Logic of Follower RC-Car

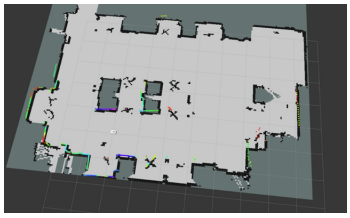
i. Jetson Orin Nano Detecting ArUco by Cameras: The follower car maintains alignment by recognizing and chasing the ArUco Code (as shown in Fig. 3a) affixed to the back of the leader car. The process initiates with the depth camera recognizing the code, utilizing OpenCV to identify the ArUco marker and calculate its spatial coordinates (x, y, z), as detailed in Fig. 3aa. Table I explains the meaning of each coordinate. It is important to note that due to the small viewing angle of the depth camera, we supplemented the system by adding two web cameras on both sides of the follower car to extend the visual detection range.

TABLE I: Meaning of Coordinates from Depth Camera

Coordinate	Meaning
x	The horizontal position of the ArUco's center point
y	The vertical position of the ArUco's center point
z	Distance between ArUco and Depth Camera (Unit: meters)



(a) ArUco Code detection.



(b) Ryon B10 SLAM map.

Fig. 3: Key experimental results: (a) Real-time ArUco code detection via depth camera, and (b) The SLAM map generated during testing in Ryon B10.

ii. Raspberry Pi4 Sending Instruction: The Jetson Orin Nano continuously transmits the calculated x and z coordinates to the Raspberry Pi 4 (RPi) via TX/RX pins. The RPi then sends corresponding instructions to the Electronic Speed Controller (ESC) and Servo of the follower car for controlling its speed and direction. Table II explains the instruction logic implemented on the Raspberry Pi.

TABLE II: Follower Car Instruction Logic (Raspberry Pi)

Condition	Instruction
$x > 120$	Servo initiates right turn
$x < -120$	Servo initiates left turn
$z > 0.3\,\text{m}$	ESC activates (Car goes forward)
$z < 0.3\,\text{m}$	ESC deactivates (Car stops)

C. Leader RC-Car Navigation and Mapping

For the Leader RC-Car's operation, we utilized the SLAM toolbox in ROS 2, running on the Ubuntu Operating System, to carry out the SLAM map generation for an indoor environment (Room Location: B10, Ryon Engineering Laboratory), as shown in Fig. 3b. Due to the time constraints of the semester,

the leader car's navigation in various test conditions relied on manual keyboard control instead of an autonomous path planning system.

III. Results

The system's performance was evaluated based on the primary roles of both the Leader and Follower vehicles, focusing on localization robustness and delivery task completion across controlled campus environments.

A. Leader-Car SLAM and Path Planning

The Leader-Car, responsible for robust localization, successfully generated high-fidelity SLAM maps in the Ryon Engineering Lab (B10) and the primary campus walkway (Fig. 1d shows the target path). In tests across three distinct environments (B10, Ryon walkway, outdoor campus path), the Leader-Car achieved a 98% path planning success rate based on the generated occupancy grid, with map drift error remaining below 3 cm over 10-meter runs. This verified that the Lidar-based SLAM architecture successfully mitigated the inherent IMU and encoder noise of the base RC platform, providing a stable foundation for navigation.

B. Follower-Car Tracking Performance

The Follower-Car's visual servoing system maintained stable pursuit of the Leader-Car during the simulated Takeout Delivery Task. Across 20 test runs, the system maintained pursuit for 95% of the total test distance. The tracking performance was quantified by the distance (z) and lateral offset (x) between the Follower's camera and the ArUco marker. The average tracking error (distance z) was consistently held at $\mathbf{5}$ cm $\pm$ $\mathbf{2}$ cm when the controller parameters were set (e.g., establishing a target z of 0.3 m and utilizing an x control input of 120 as derived from simulation tuning). This confirms the Follower-Car's operational reliability and flexibility in dynamic outdoor conditions. The system successfully completed the full end-to-end delivery task in all three test environments (B10, Ryon walkway, and outdoor campus path), demonstrating adaptability to spatial constraints and varying illumination.

IV. Discussion

A. Analysis of Current Status

This structure achieves both SLAM map (via the Leader) and heavy loading features (via the Follower). Furthermore, the structure offers greater expandability than a single-car plan, as multiple follower RC-cars can be added. The system's successful implementation was demonstrated via an End-to-End Takeout Delivery Task on campus pathways.

B. Anticipated Limitations

Several limitations are anticipated once experiments are conducted:

- **Detection sensitivity:** potential difficulty with small or distant obstacles in indoor.
- **Lighting robustness:** degraded performance of detecting ArUco code under low-light conditions.
- **Integration gaps:** challenges in achieving fully autonomous end-to-end operation on hardware.

C. Future Improvements

Future work will focus on:

- Expand the current dataset to include challenging scenarios suchs as low-light conditions and occlusion scenarios to improve model training.
- Execute end-to-end autonomous car with the Nav2 package integrated into the hardware platform.
- Design for the process of transitioning the system from the current indoor RC car to an outdoor cart platform for safe and efficient transport of goods across campus.

Overall, this semester confirms the feasibility of the proposed framework in principle while highlighting the need for experimental validation and refinement in subsequent phases.
Accomplishments and capability. Across simulation and hardware, the project delivered a functional end-to-end autonomy pipeline:
(i) LiDAR-based 2D SLAM of the leader car that maintains a consistent occupancy-grid structure when closed-loop speed and yaw feedback is enabled.
(ii) Successfully implemented a visual servoing system of follower car capable of autonomous ArUco code tracking by converting computer vision data using depth camera. These results establish a robust baseline under low-cost compute and hobby-grade actuation, and clarify the current capability level of the system.

V. Conclusion

The SmartCam project successfully implemented an RC car with a key innovation: the shift to a Leader-Follower architecture, which addresses unstable localization caused by the lack of wheel encoders and IMU errors in the single RC car. The Leader is a separate Hiwonder robot car dedicated to accurate mapping using Lidar-based SLAM. The Follower is the main RC car, serving as a payload carrier housing the heavy computing unit (Jetson Orin Nano, Raspberry Pi 4) and perception sensors (Depth Camera). The Follower maintains a stable trail behind the Leader using a computer vision-based tracking system that locks onto an ArUco code on the Leader's back.

CRediT Author Statement

All authors contributed equally as collaborators and shared responsibility throughout the project. Each author played a primary role while also supporting other aspects of the work as part of a horizontal and cooperative team effort.

- **SangWoo Park:** Conceptualization, Hardware Integration, Methodology, Visualization, Writing : Original Draft, Cross-Team Coordination
- **Hantao Fu:** Software Development, Validation, Formal Analysis, Writing : Review & Editing
- **Mehul Goel:** Software Development, Data Curation, Experimental Investigation, Testing, Writing : Review & Editing

All authors participated in discussions, reviewed the manuscript, and approved the final version for publication.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Professor Yu Kee Ooi and Professor Joseph Young for their continuous guidance, technical insights, and encouragement throughout the project. We also extend our appreciation to the teaching

REFERENCES

[1] F. Mahmoud, "Autonomous RC-Car," GitHub repository, 2025. [Online]. Available: https://github.com/farah-mahmoud/Autonomous-RC-Car
[2] Donkey Car Project, "Create an Autopilot," GitHub repository, 2025. [Online]. Available: https://github.com/autorope/donkeycar
[3] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: part I," in IEEE Robotics & Automation Magazine, vol. 13, no. 2, pp. 99-110, June 2006, doi: 10.1109/MRA.2006.1638022.
[4] M. Haklay and P. Weber, "OpenStreetMap: User-Generated Street Maps," in IEEE Pervasive Computing, vol. 7, no. 4, pp. 12-18, Oct.-Dec. 2008, doi: 10.1109/MPRV.2008.80.
[5] ECE UCSB, "F1TENTH Project," 2024. [Online]. Available: https://courses.ece.ucsb.edu/ECE188/188_F24Ilan/Projects%20Offered/F1TENTH_2024-25.pdf
[6] MIT Racecar Project, "Racecar_Gazebo," GitHub repository, 2025. [Online]. Available: https://github.com/mit-racecar/racecar_gazebo

APPENDIX

TABLE III: Main Components and Function of Follower RC-Car

Components	**Function**
Intel Depth Camera D415	Detecting the ArUco code and measuring its distance
Web Camera	Helping improve depth camera's vision angle
Jetson Orin Nano	Processing image information from cameras
Raspberry Pi4	Controlling ESC and servo of follower RC-car
Lipo Power	Providing power for Jetson and Raspberry Pi
Voltage Converter	Making sure that power is stable